

K.R. Mangalam University

School of Engineering and Technology

Task -1 Execution time, memory usage, and complexity comparison table for all implemented Greedy algorithms.

Program: BCA (AI & DS) Course Code - ENCA351 (2026-27)

Lab 3: Smart Resource Allocation and Scheduling System using Greedy Algorithms

Submitted by:

Name: Bhavya Rattan
RollNumber: 2401201004

Course: BCA (AI & DS)
Section: B

Submitted To:

Dr. Aarti Sangwan

Assistant Professor

1. Objective

Implement an Automated Benchmarking Engine that executes the three greedy algorithms of Lab 3 for multiple input sizes and records execution time, memory usage and operation counts, and then compares the measured behaviour with the theoretical time and space complexity. The engine is written in Python 3 (file: `analysis/generate_benchmarks_and_charts.py`) and is automated: one command (`python analysis/generate_benchmarks_and_charts.py`) runs all nine input sizes, saves the results to `data_csv/benchmark_results.csv` and regenerates the charts in the `graphes/` folder. The same algorithms are also used by the menu-driven application (`app.py`) and the Jupyter notebook (`notebooks/project_notebook.ipynb`) in the repository.

2. Design of the engine

Component	Purpose / technique
run_benchmarks()	Generic runner. For each input size n it builds a fresh random input for every algorithm (seeded with random.seed(42) so results are reproducible), measures time, measures peak memory and appends one record per size (n, time, memory for each algorithm) to a pandas DataFrame, which is exported to CSV.
Algorithms under test	Three greedy algorithms (algorithms/ package): Fractional Knapsack (resource allocation), Activity Selection (task scheduling) and Huffman Coding (data compression) – 3 programs in total.
Time measurement	time.perf_counter() is read immediately before and after each call; the elapsed time is stored in seconds and shown in ms in this report. Each (algorithm, n) pair is timed on one run on a freshly generated input.
Memory measurement	tracemalloc: tracing is started just before the algorithm call and stopped after it, and the peak traced bytes are reported in KB. The input is built before tracing starts, so the figure is the extra memory the algorithm itself allocates (sorted copy, selected list, heap, codebook, encoded bit-string).
Operation counts	Basic operations = key comparisons made by the sort (Knapsack: value/weight ratio, Activity: finish time) or by the min-heap (Huffman), plus the iterations of the main greedy loop (Huffman also counts the one-pass frequency scan, the n-1 heap merges and the n code look-ups of encoding). The CSV does not store operations, so they were obtained from an instrumented run using the same input generators and seed.
Theoretical complexity	Fractional Knapsack, Activity Selection and Huffman Coding are all O(n log n) in time and O(n) in space (for Huffman, n is the number of distinct characters). The empirical growth is compared with this prediction through the log-log slope.
Persistence and plots	The runner writes data_csv/benchmark_results.csv and generate_performance_and_memory_charts() saves performance_comparison.png and memory_usage_comparison.png in graphes/.

3. Test configuration

Algorithm	Input sizes tested (n)	Input type	Greedy criterion
Fractional Knapsack	10; 50; 100; 250; 500; 1,000; 2,500; 5,000; 10,000	n random items, weight in [5,100], value in [10,500]; capacity = 25n	Highest value-to-weight ratio
Activity Selection	10; 50; 100; 250; 500; 1,000; 2,500; 5,000; 10,000	n random activities, start in [0,2n], duration 1–50	Earliest finish time first
Huffman Coding	10; 50; 100; 250; 500; 1,000; 2,500; 5,000; 10,000	Text of n characters from 62 letters/digits with skewed frequencies (weight 1/(i+1))	Merge two lowest-frequency nodes

Total: 3 algorithms × 9 input sizes = 27 measurements (exported to benchmark_results.csv, 9 rows with 3 time and 3 memory columns).

4. Sample output (console)

Running `python analysis/generate_benchmarks_and_charts.py` executes the benchmark for all nine input sizes, exports the dataset to benchmark_results.csv and regenerates the charts. The exported dataset (time in ms) is shown below:

```
$ python analysis/generate_benchmarks_and_charts.py
[*] Running comprehensive benchmark experiments for all 3 greedy algorithms...
    Benchmarking size n = 10 ...
    Benchmarking size n = 50 ...
    Benchmarking size n = 100 ...
    Benchmarking size n = 250 ...
    Benchmarking size n = 500 ...
    Benchmarking size n = 1000 ...
    Benchmarking size n = 2500 ...
    Benchmarking size n = 5000 ...
    Benchmarking size n = 10000 ...
[OK] Benchmark results saved to data_csv/benchmark_results.csv

$ cat data_csv/benchmark_results.csv   # columns shown as a table

      n |   Knap ms   Knap KB |    Act ms    Act KB |   Huff ms   Huff KB
--------------------------------------------------------------------------
     10 |    0.1191      2.75 |    0.0859      2.48 |    0.3043      2.92
     50 |    0.3578      7.41 |    0.3858     10.68 |    0.5298      8.00
    100 |    0.6755     14.35 |    0.7214     20.27 |    0.7476     10.97
    250 |    2.4506     41.91 |    1.9168     54.14 |    1.1844     19.46
    500 |    3.7750     82.68 |    4.0090    110.04 |    1.4236     23.88
  1,000 |    8.0878    166.93 |    8.2191    223.14 |    1.7599     31.75
  2,500 |   15.2443    431.63 |   13.9410    564.02 |    1.8480     50.90
  5,000 |   48.2021    836.10 |   36.2427   1126.12 |    2.9401     84.16
 10,000 |  140.2810   1923.62 |   60.1385   2261.45 |    5.9603    151.36
```

Figure 1: Console output of the benchmarking engine and the exported benchmark_results.csv.

5. Summary of recorded measurements (largest input of each algorithm)

Algorithm	n	Time (ms)	Memory (KB)	Operations	Time / Space
Fractional Knapsack	10,000	140.2810	1,923.62	126,102	O(n log n) / O(n)
Activity Selection	10,000	60.1385	2,261.45	130,126	O(n log n) / O(n)
Huffman Coding	10,000	5.9603	151.36	20,787	O(n log n) / O(n)

Memory is the peak allocation seen by tracemalloc while the algorithm runs (the input itself is built before tracing starts). Knapsack and Activity Selection allocate a sorted copy and a result list, so memory grows linearly with n. Huffman Coding is dominated by the alphabet size (at most 62 distinct characters) plus the encoded bit-string, so its memory grows much more slowly.

6. Complexity comparison table (theoretical vs observed)

The table compares the theoretical time and space complexity of each implemented greedy algorithm with the values observed by the benchmarking engine (benchmark_results.csv, 27 data points). The observed slope is the gradient of log(time) against log(n); for an O(n log n) algorithm it should be slightly above 1.

Algorithm	Theoretical time	Theoretical space	Input range tested	Time at max n (ms)	Ops at max n	Peak mem at max n (KB)	Observed slope (time)	Growth trend observed
Fractional Knapsack	O(n log n)	O(n)	10 – 10,000	140.2810	126,102	1,923.62	1.01 (theory ~1.1)	Near-linear: n grows 1,000× (10 to 10,000) and time grows about 1,178×. Operation count rises 32 to 126,102 (n log n shape); memory slope 0.98.
Activity Selection	O(n log n)	O(n)	10 – 10,000	60.1385	130,126	2,261.45	0.96 (theory ~1.1)	Near-linear: time grows about 700× for 1,000× input. Operations rise 35 to 130,126 (sort on finish time dominates); memory slope 1.00.
Huffman Coding	O(n log n)	O(n)	10 – 10,000	5.9603	20,787	151.36	0.39 (theory ~1, see note)	Sub-linear in n: time grows only about 20×. The heap work depends on the 62 distinct characters (saturated from n = 2,500), so only the linear text scan keeps growing; memory slope 0.54.

** Space column is the theoretical auxiliary space: all three algorithms are O(n) because they build a sorted copy / selected list (Knapsack, Activity) or a tree and codebook (Huffman, where n is the number of distinct characters).*

Observed slope = gradient of log(time) vs log(n) fitted over all 9 input sizes; a slope of k means time grows proportionally to n^k. An n log n curve fitted over 10 – 10,000 gives a slope a little above 1, but fixed per-call overhead at small n pulls the measured slope down to about 1. For Huffman Coding the theoretical n is the number of distinct characters (capped at 62 here), while the benchmark varies the text length, so its measured slope is below 1. Each timing is a single run, so small deviations are expected.

7. Observations

• **Fractional Knapsack** and **Activity Selection** behave as predicted: both are dominated by a sort, the operation count follows n log n (about 126 thousand and 130 thousand comparisons-plus-iterations at n = 10,000) and time and memory grow almost linearly (slopes 1.01 and 0.96 for time).

• At n = 10,000 Knapsack took 140.28 ms and Activity Selection 60.14 ms. Knapsack is slower mainly because it creates a KnapsackItem object (with a ratio computation) for every item and sorts on a float key.

• **Huffman Coding** is the fastest and lightest (5.96 ms, 151.36 KB at n = 10,000) because its heap only holds the distinct characters, not every input symbol.

• All three algorithms are O(n log n) in time and O(n) in space, and the benchmark data support this: no algorithm showed super-linear growth beyond the n log n trend.

8. Performance and memory charts

The charts below are generated by the same engine from benchmark_results.csv (graphes/ folder).

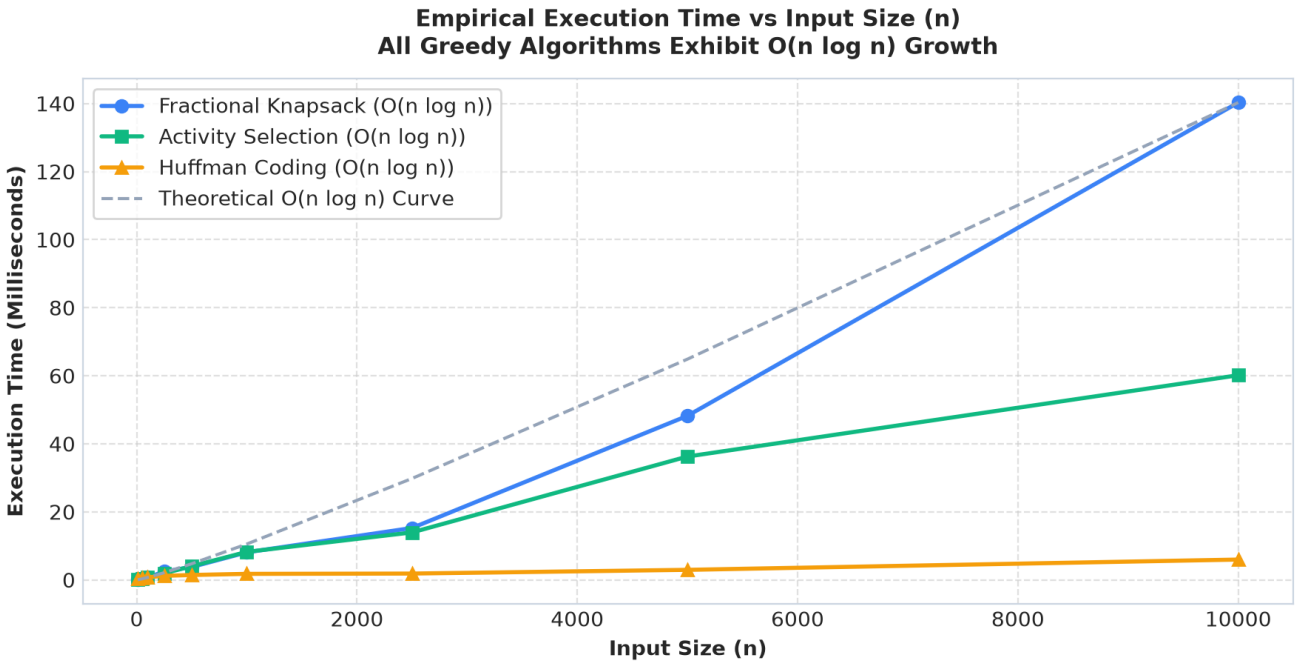


Figure 2: Execution time vs input size for the three greedy algorithms.

Bhavya Rattan (Roll No: 2401201004) | DAA Lab 3 Benchmarks

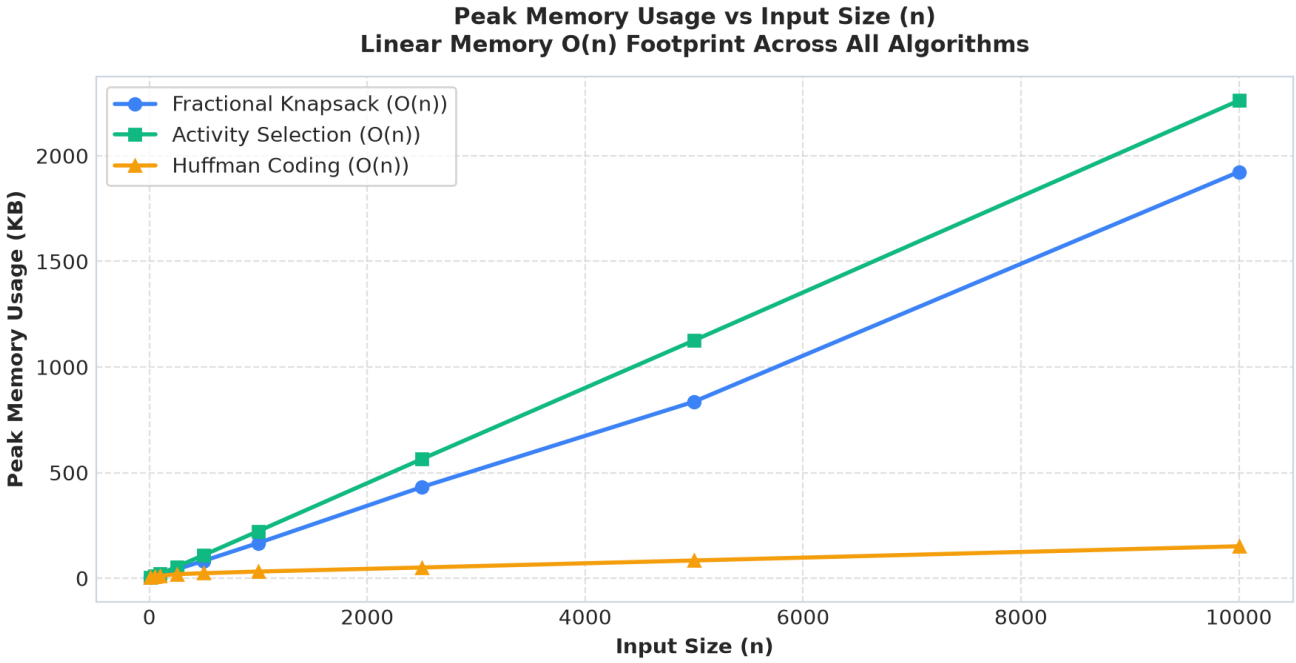


Figure 3: Peak memory usage vs input size for the three greedy algorithms.

Appendix: complete source code

algorithms/fractional_knapsack.py

```
"""
Resource Allocation Module: Fractional Knapsack Algorithm
Greedy Strategy: Value-to-Weight Ratio Maximization
Time Complexity: O(n log n)
Space Complexity: O(n)
Author: Bhavya Rattan (Roll No: 2401201004)
"""

from typing import List, Dict, Tuple, Union


class KnapsackItem:
    def __init__(self, name: str, weight: float, value: float):
        self.name = name
        self.weight = float(weight)
        self.value = float(value)
        self.ratio = self.value / self.weight if self.weight > 0 else 0.0

    def __repr__(self):
        return f"Item({self.name}, w={self.weight}, v={self.value}, ratio={self.ratio:.2f})"


def fractional_knapsack(
    items: List[Union[Dict, KnapsackItem]],
    capacity: float
) -> Tuple[float, List[Tuple[str, float, float, float]]]:
    """
    Computes the maximum achievable profit using Fractional Knapsack algorithm.

    Parameters:
        items: List of item dictionaries (with 'name', 'weight', 'value') or KnapsackItem instances.
        capacity: Maximum weight capacity of the knapsack.

    Returns:
        total_value: Maximum total value/profit achievable.
        allocation: List of tuples (item_name, fraction_taken, weight_taken, value_contributed).
    """
    if capacity <= 0 or not items:
        return 0.0, []

    # Standardize to list of KnapsackItem objects
    parsed_items: List[KnapsackItem] = []
    for it in items:
        if isinstance(it, KnapsackItem):
            parsed_items.append(it)
        else:
            parsed_items.append(KnapsackItem(it["name"], it["weight"], it["value"]))

    # Greedy Choice: Sort items in DESCENDING order of value-to-weight ratio
    sorted_items = sorted(parsed_items, key=lambda x: x.ratio, reverse=True)

    total_value = 0.0
    allocation: List[Tuple[str, float, float, float]] = []
    remaining_capacity = float(capacity)

    for item in sorted_items:
        if remaining_capacity <= 0:
            break

        if item.weight <= remaining_capacity:
            # Take the entire item (fraction = 1.0)
            fraction = 1.0
            weight_taken = item.weight
            val_taken = item.value
            total_value += val_taken
            remaining_capacity -= weight_taken
            allocation.append((item.name, fraction, weight_taken, val_taken))
        else:
            # Take a fraction of the item to fill the remaining capacity
            fraction = remaining_capacity / item.weight
            weight_taken = remaining_capacity
            val_taken = item.value * fraction
            total_value += val_taken
            allocation.append((item.name, fraction, weight_taken, val_taken))
            remaining_capacity = 0.0

    return total_value, allocation


if __name__ == "__main__":
    test_items = [
        {"name": "Gold", "weight": 10, "value": 60},
        {"name": "Silver", "weight": 20, "value": 100},
        {"name": "Bronze", "weight": 30, "value": 120},
```

```
    ]
    cap = 50
    max_val, details = fractional_knapsack(test_items, capacity=cap)
    print(f"--- Fractional Knapsack Allocation (Capacity: {cap}) ---")
    print(f"Maximum Achievable Profit: {max_val:.2f}")
    for name, frac, wt, val in details:
        print(f"  {name}: {frac*100:.1f}% taken | Weight: {wt:.1f} | Value: {val:.1f}")
```

algorithms/activity_selection.py

```
"""
Task Scheduling Module: Activity Selection Algorithm
Greedy Strategy: Earliest Finish Time First
Time Complexity: O(n log n)
Space Complexity: O(n)
Author: Bhavya Rattan (Roll No: 2401201004)
"""

from typing import List, Dict, Union


class Activity:
    def __init__(self, name: str, start: float, finish: float):
        self.name = name
        self.start = float(start)
        self.finish = float(finish)
        self.duration = self.finish - self.start

    def __repr__(self):
        return f"Activity({self.name}, [{self.start} -> {self.finish}], dur={self.duration})"


def activity_selection(activities: List[Union[Dict, Activity]]) -> List[Dict]:
    """
    Selects the maximum number of mutually compatible, non-overlapping activities.

    Parameters:
        activities: List of activity dicts (with 'name', 'start', 'finish') or Activity instances.

    Returns:
        List of selected activity dictionaries sorted in execution order.
    """
    if not activities:
        return []

    # Standardize input to list of Activity objects
    parsed_acts: List[Activity] = []
    for a in activities:
        if isinstance(a, Activity):
            parsed_acts.append(a)
        else:
            parsed_acts.append(Activity(a["name"], a["start"], a["finish"]))

    # Greedy Choice: Sort activities in ASCENDING order of FINISH times
    sorted_acts = sorted(parsed_acts, key=lambda a: a.finish)

    selected: List[Dict] = []
    # Always select the first activity (earliest finish time)
    first = sorted_acts[0]
    selected.append({"name": first.name, "start": first.start, "finish": first.finish, "duration": first.duration})
    last_finish = first.finish

    # Iterate through the rest and pick compatible activities
    for act in sorted_acts[1:]:
        if act.start >= last_finish:
            selected.append({"name": act.name, "start": act.start, "finish": act.finish, "duration": act.duration})
            last_finish = act.finish

    return selected


if __name__ == "__main__":
    sample_activities = [
        {"name": "A1", "start": 1, "finish": 4},
        {"name": "A2", "start": 3, "finish": 5},
        {"name": "A3", "start": 0, "finish": 6},
        {"name": "A4", "start": 5, "finish": 7},
        {"name": "A5", "start": 3, "finish": 9},
        {"name": "A6", "start": 5, "finish": 9},
        {"name": "A7", "start": 6, "finish": 10},
        {"name": "A8", "start": 8, "finish": 11},
    ]
    scheduled = activity_selection(sample_activities)
    print(f"--- Task Scheduling: Selected {len(scheduled)} Activities ---")
    for act in scheduled:
        print(f"  {act['name']}: Start={act['start']}, Finish={act['finish']} (Duration={act['duration']})")
```

algorithms/huffman_coding.py

```
"""
Data Compression Module: Huffman Coding Algorithm
Greedy Strategy: Frequency-based Minimum Priority Queue Merging (Prefix-Free Codes)
Time Complexity: O(n log n) where n is distinct characters
Space Complexity: O(n)
Author: Bhavya Rattan (Roll No: 2401201004)
"""

import heapq
from collections import Counter
from typing import Dict, Tuple, Optional


class Node:
    """Represents a node in the Huffman Tree."""
    def __init__(self, char: Optional[str], freq: int, left: Optional['Node'] = None, right: Optional['Node'] = None):
        self.char = char
        self.freq = freq
        self.left = left
        self.right = right

    def __lt__(self, other: 'Node') -> bool:
        # Min-heap comparison based on frequency
        return self.freq < other.freq

    def is_leaf(self) -> bool:
        return self.left is None and self.right is None


def build_huffman_tree(text: str) -> Optional[Node]:
    """Builds the optimal Huffman Tree using min-heap priority queue."""
    if not text:
        return None

    # Count character frequencies
    frequency = Counter(text)

    # Initialize min-heap with leaf nodes for all distinct characters
    heap = [Node(char=ch, freq=freq) for ch, freq in frequency.items()]
    heapq.heapify(heap)

    # Handle edge case where text contains only one unique character
    if len(heap) == 1:
        single = heapq.heappop(heap)
        return Node(char=None, freq=single.freq, left=single, right=None)

    # Greedily merge two nodes with the lowest frequencies
    while len(heap) > 1:
        left = heapq.heappop(heap)
        right = heapq.heappop(heap)
        merged = Node(char=None, freq=left.freq + right.freq, left=left, right=right)
        heapq.heappush(heap, merged)

    return heap[0]  # Root of the Huffman Tree


def generate_codes(node: Optional[Node], prefix: str = "", codebook: Optional[Dict[str, str]] = None) -> Dict[str, str]:
    """Traverses Huffman tree to generate variable-length prefix-free binary codes."""
    if codebook is None:
        codebook = {}
    if node is None:
        return codebook

    if node.is_leaf():
        codebook[node.char] = prefix if prefix else "0"
        return codebook

    if node.left:
        generate_codes(node.left, prefix + "0", codebook)
    if node.right:
        generate_codes(node.right, prefix + "1", codebook)

    return codebook


def huffman_encode(text: str) -> Tuple[str, Dict[str, str], Dict[str, int], Dict[str, float]]:
    """
    Encodes text using Huffman Coding and calculates complete compression statistics.

    Returns:
        encoded_bits: The compressed binary string.
        codebook: Mapping of character -> binary code.
        frequencies: Character frequency counts.
        stats: Detailed statistics dictionary.
    """
    if not text:
        return "", {}, {}, {"original_bits": 0, "compressed_bits": 0, "ratio": 0.0, "savings": 0.0}

    root = build_huffman_tree(text)
```

```
    codebook = generate_codes(root)
    frequencies = dict(Counter(text))

    encoded_bits = "".join(codebook[ch] for ch in text)
    original_bits = len(text) * 8  # Standard ASCII 8 bits per character
    compressed_bits = len(encoded_bits)
    compression_ratio = (compressed_bits / original_bits) if original_bits > 0 else 0.0
    space_savings = (1.0 - compression_ratio) * 100.0 if original_bits > 0 else 0.0
    avg_code_length = (compressed_bits / len(text)) if len(text) > 0 else 0.0

    stats = {
        "original_chars": len(text),
        "distinct_chars": len(frequencies),
        "original_bits": original_bits,
        "compressed_bits": compressed_bits,
        "compression_ratio": compression_ratio,
        "space_savings_percent": space_savings,
        "avg_code_length": avg_code_length,
    }

    return encoded_bits, codebook, frequencies, stats


def huffman_decode(encoded_bits: str, root: Node) -> str:
    """Decodes the binary bitstream back into the original text string."""
    if not encoded_bits or not root:
        return ""

    decoded_chars = []
    curr = root

    for bit in encoded_bits:
        if bit == "0":
            curr = curr.left if curr.left else curr
        else:
            curr = curr.right if curr.right else curr

        if curr.is_leaf():
            decoded_chars.append(curr.char)
            curr = root

    return "".join(decoded_chars)


if __name__ == "__main__":
    sample_text = "abracadabra"
    bits, codes, freqs, statistics = huffman_encode(sample_text)
    print(f"--- Huffman Coding Results for '{sample_text}' ---")
    print("Character Frequencies:", freqs)
    print("Huffman Codebook:", codes)
    print("Encoded Bitstream:", bits)
    print(f"Original Bits: {statistics['original_bits']} | Compressed Bits: {statistics['compressed_bits']}")
    print(f"Compression Ratio: {statistics['compression_ratio']:.2%} | Space Savings: {statistics['space_savings_percent']:.2f}%")

    # Verify lossless decoding
    tree_root = build_huffman_tree(sample_text)
    decoded = huffman_decode(bits, tree_root)
    print("Decoded String:", decoded)
    assert decoded == sample_text, "Decoding failed!"
    print("Lossless decompression verified successfully!")
```

analysis/generate_benchmarks_and_charts.py (benchmark engine: run_benchmarks)

```
def run_benchmarks():
    print("[*] Running comprehensive benchmark experiments for all 3 greedy algorithms...")

    input_sizes = [10, 50, 100, 250, 500, 1000, 2500, 5000, 10000]
    results = []

    for n in input_sizes:
        print(f"    Benchmarking size n = {n} ...")

        # ------------------------------
        # 1. Fractional Knapsack Benchmark
        # ------------------------------
        knapsack_items = [
            {"name": f"Item_{i}", "weight": random.uniform(5, 100), "value": random.uniform(10, 500)}
            for i in range(n)
        ]
        cap = n * 25.0

        tracemalloc.start()
        t0 = time.perf_counter()
        fractional_knapsack(knapsack_items, capacity=cap)
        t_knapsack = time.perf_counter() - t0
        _, mem_knapsack = tracemalloc.get_traced_memory()
        tracemalloc.stop()

        # ------------------------------
```

```
        # 2. Activity Selection Benchmark
        # ----------------------------
        activities = []
        for i in range(n):
            s = random.randint(0, n * 2)
            f = s + random.randint(1, 50)
            activities.append({"name": f"A_{i}", "start": s, "finish": f})

        tracemalloc.start()
        t0 = time.perf_counter()
        activity_selection(activities)
        t_activity = time.perf_counter() - t0
        _, mem_activity = tracemalloc.get_traced_memory()
        tracemalloc.stop()

        # ----------------------------
        # 3. Huffman Coding Benchmark
        # ----------------------------
        # Generate text with skewed frequency distribution
        charset = string.ascii_letters + string.digits
        weights = [1.0 / (i + 1) for i in range(len(charset))]
        text_sample = "".join(random.choices(charset, weights=weights, k=n))

        tracemalloc.start()
        t0 = time.perf_counter()
        huffman_encode(text_sample)
        t_huffman = time.perf_counter() - t0
        _, mem_huffman = tracemalloc.get_traced_memory()
        tracemalloc.stop()

        results.append({
            "Input_Size_n": n,
            "Knapsack_Time_sec": t_knapsack,
            "Knapsack_Mem_KB": mem_knapsack / 1024.0,
            "Activity_Time_sec": t_activity,
            "Activity_Mem_KB": mem_activity / 1024.0,
            "Huffman_Time_sec": t_huffman,
            "Huffman_Mem_KB": mem_huffman / 1024.0,
        })

    df = pd.DataFrame(results)
    csv_path = os.path.join(DATA_DIR, "benchmark_results.csv")
    df.to_csv(csv_path, index=False)
    print(f"[OK] Benchmark results saved to {csv_path}")

    return df


# ============================================================================
```